

Column Transformer

What's Covered

1. The Problem — Mixed Column Types in One Dataset
2. What is ColumnTransformer?
3. Basic Syntax & Structure
4. Worked Example — Step by Step
5. The remainder Parameter
6. ColumnTransformer Inside a Pipeline
7. Selecting Columns by Name vs Index vs dtype
8. Common Pitfalls
9. Summary

1. The Problem — Mixed Column Types in One Dataset

Real-world datasets almost never contain just one type of column. A single dataset might have numerical columns that need scaling, ordinal columns that need OrdinalEncoder, nominal columns that need One-Hot Encoding, and columns with missing values that need imputation — all at the same time.

Without a tool to organise this, you'd have to manually slice out each column type, transform it separately, and then carefully stitch the results back together in the right order — tedious, error-prone, and hard to reuse on new data. ColumnTransformer automates this entire process.

1.1 Example Mixed Dataset

Column	Type	Needs
Age	Numerical	StandardScaler / MinMaxScaler
Fever	Numerical (with missing)	SimpleImputer then scaling
Gender	Nominal categorical	OneHotEncoder
City	Nominal categorical	OneHotEncoder
Cough	Ordinal categorical	OrdinalEncoder

2. What is ColumnTransformer?

Definition: ColumnTransformer is a scikit-learn tool that applies different preprocessing transformations to different subsets of columns in a single dataset, and then automatically combines all the results into one unified output array.

Instead of writing separate code to transform each column group and manually concatenating results, you describe each transformation once, and ColumnTransformer handles fitting, transforming, and combining everything consistently — for both training and test data.

2.1 Key Benefits

- Single object to fit and transform — no manual column slicing.
- Keeps training and test transformations perfectly consistent.
- Integrates seamlessly into a scikit-learn Pipeline.
- Makes preprocessing code shorter, cleaner, and reusable.

3. Basic Syntax & Structure

ColumnTransformer takes a list of tuples. Each tuple defines one transformation: a name, the transformer object, and the list of columns it should apply to.

```
ColumnTransformer([ ('name', transformer, [columns]), ... ])
```

Tuple Element	Meaning
name	A string label you choose to identify this transformation step
transformer	Any sklearn transformer object — e.g. StandardScaler(), OneHotEncoder()
[columns]	A list of column names (or indices) this transformer should apply to

```
# Import the tool
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

# Define which transformation applies to which columns
ct = ColumnTransformer(transformers=[
    ('scaling', StandardScaler(), ['Age', 'Fever']),
    ('encoding', OneHotEncoder(), ['Gender', 'City'])
])
```

Python

4. Worked Example — Step by Step

Applying three different preprocessing steps to three different column groups of a small medical dataset.

4.1 Raw Data

Age	Fever	Gender	City	Cough	Has_Disease
25	98.6	Male	Mumbai	Mild	No
40	101.2	Female	Delhi	Strong	Yes
33	NaN	Female	Pune	Mild	No
55	99.5	Male	Mumbai	Strong	Yes

4.2 Define the Transformer

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import (StandardScaler,
                                   OneHotEncoder, OrdinalEncoder)

# Fever has missing values -> impute first, then scale
from sklearn.pipeline import Pipeline
fever_pipe = Pipeline([
    ('impute', SimpleImputer(strategy='mean')),
    ('scale', StandardScaler())
])

# Build the full ColumnTransformer
ct = ColumnTransformer(transformers=[
    ('age_scale', StandardScaler(), ['Age']),
    ('fever_pipe', fever_pipe, ['Fever']),
    ('gender_city', OneHotEncoder(drop='first'), ['Gender', 'City']),
    ('cough_ord', OrdinalEncoder(categories=[['Mild', 'Strong']]), ['Cough'])
])
```

Python

4.3 Fit and Transform

```
# Fit on training data – learns statistics/categories for ALL steps at once
X_train_transformed = ct.fit_transform(X_train)

# Transform test data using the SAME fitted transformations
X_test_transformed = ct.transform(X_test)

# View the generated column names for all combined transformers
print(ct.get_feature_names_out())
```

Python

Notice: a single `fit_transform()` call ran the `StandardScaler` on `Age`, the `impute+scale Pipeline` on `Fever`, `OneHotEncoder` on `Gender` and `City`, and `OrdinalEncoder` on `Cough` — then automatically concatenated all results into one combined array.

5. The remainder Parameter

By default, any column NOT mentioned in the transformers list is DROPPED from the output. The `remainder` parameter controls this behaviour.

remainder Value	Behaviour
'drop' (default)	Columns not listed in any transformer are removed from the output
'passthrough'	Columns not listed are kept AS-IS, unchanged, appended at the end
some transformer	e.g. <code>StandardScaler()</code> — applies that transformer to all remaining columns

```
# Keep any leftover columns unchanged instead of dropping them
ct = ColumnTransformer(transformers=[
    ('encoding', OneHotEncoder(), ['Gender', 'City'])
], remainder='passthrough')

# Now 'Age', 'Fever', 'Cough' pass through untouched
# and are appended after the encoded columns
```

Python

6. ColumnTransformer Inside a Pipeline

ColumnTransformer is most powerful when combined with a Pipeline — preprocessing and the model become a single object that can be fit, transformed, and used to predict in one call, with zero risk of data leakage.

```
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

# Combine preprocessing (ColumnTransformer) and model
full_pipeline = Pipeline([
    ('preprocessing', ct),
    ('model', LogisticRegression())
])

# fit() runs ColumnTransformer.fit_transform, then model.fit
full_pipeline.fit(X_train, y_train)

# predict() runs ColumnTransformer.transform, then model.predict
predictions = full_pipeline.predict(X_test)

# score() works the same way – fully consistent end-to-end
accuracy = full_pipeline.score(X_test, y_test)
```

Python

This is the standard professional pattern: ColumnTransformer handles all column-specific preprocessing, and the outer Pipeline chains it directly into the model — one fit() call does everything safely, every time.

7. Selecting Columns by Name vs Index vs dtype

Selection Method	Example
By column name (list of strings)	['Age', 'Fever'] — works when input is a DataFrame
By column index (list of integers)	[0, 1] — works for both arrays and DataFrames
By dtype using make_column_selector	make_column_selector(dtype_include='object') — auto-selects all text/categorical columns

```
# Automatically select columns by data type instead of naming them manually
from sklearn.compose import make_column_selector

ct = ColumnTransformer(transformers=[
    ('num', StandardScaler(),
     make_column_selector(dtype_include='number')),
    ('cat', OneHotEncoder(),
     make_column_selector(dtype_include='object'))
])
```

Python

`make_column_selector` is convenient for datasets with many columns — it removes the need to type out every column name and automatically adapts if columns are added or removed later.

8. Common Pitfalls

- ■ **Forgetting that unmentioned columns are dropped by default:** Set `remainder='passthrough'` if you want to keep them unchanged.
- ■ **Fitting ColumnTransformer on the full dataset before splitting:** Standard data leakage mistake — fit only on training data.
- ■ **Mismatched column lists between training and new data:** If a column is renamed or missing in new data, `transform()` will fail — keep column names consistent.
- ■ **Using column indices when column order might change:** Prefer column names (or `make_column_selector`) over hard-coded indices for more robust code.
- ■ **Not wrapping imputation + scaling together for a single column:** If a numeric column has missing values AND needs scaling, use a small Pipeline (`impute -> scale`) as the transformer for that column.
- ■ **Calling `fit_transform()` on test data:** Always use `transform()` on test/new data — `fit_transform()` is for training data only.

9. Summary

Concept	Key Takeaway
Problem it solves	Different columns need different preprocessing in the same dataset
Core idea	Apply different transformers to different column subsets, then combine results
Syntax	<code>ColumnTransformer([('name', transformer, [columns]), ...])</code>
<code>fit_transform()</code>	Fits and applies all transformers in one call — training data only
<code>transform()</code>	Applies the SAME fitted transformers — use on test/new data
<code>remainder='drop'</code>	Default — columns not listed are removed from output
<code>remainder='passthrough'</code>	Keeps unlisted columns unchanged, appended at the end
Nested Pipelines	A Pipeline (e.g. <code>impute -> scale</code>) can be used as a single transformer
<code>make_column_selector</code>	Auto-selects columns by dtype instead of naming them manually
Best practice	Combine with an outer Pipeline: preprocessing (ColumnTransformer) + model
Fit rule	Always fit on training data only, transform on test data

Notes by: Amol Jagtap | amoljagtap3001@gmail.com | Day 28 — Column Transformer